

Assignment-10.5

Name:D.Divya Sri

HTNO:2303A51323

BTNO:19

Task 1:

Prompt:

Improve the readability of the following Python code by renaming unclear function and variable names to meaningful and descriptive names.

Do not change the logic of the program.

Return only the improved code.

```
def f(a, b):
```

```
    return a + b
```

```
print(f(10, 20))
```

```
#Task 1
def add_numbers(num1, num2):
    """
    Add two numbers together.

    Parameters
    -----
    num1 : int or float
        The first number to be added.
    num2 : int or float
        The second number to be added.

    Returns
    -----
    int or float
        The sum of num1 and num2.
    """
    return num1 + num2
print(add_numbers(10, 20))
```

Result:

30

Observation:

The code defines a function `add_numbers` that adds two numbers and returns the result. It has clear variable names and a short description. When called with 10 and 20, it prints 30.

Task:2

Prompt:

Improve the following Python code by adding proper error handling.
Handle possible exceptions (such as division by zero) using try-except blocks.
Display clear and user-friendly error messages.
Do not change the main logic of the program.
Return only the improved code.

```
#Task 2
def divide_numbers(numerator, denominator):
    """
    Divide one number by another with error handling.

    Parameters
    -----
    numerator : int or float
        The number to be divided.
    denominator : int or float
        The number to divide by.

    Returns
    -----
    int or float
        The result of the division if successful.
    str
        An error message if division by zero is attempted.
    """
    try:
        result = numerator / denominator
        return result
    except ZeroDivisionError:
        return "Error: Cannot divide by zero."
print(divide_numbers(10, 0))
```

```
Error: Cannot divide by zero.
```

Observation:

The code defines a function `divide_number` that performs division with proper error handling. It uses a try-except block to catch a `ZeroDivisionError` and returns a clear error message if division by zero is attempted. When called with 10 and 0, it safely prints an error message instead of crashing.

Task 3:

Prompt:

Refactor the following Python program to improve readability, structure, and code quality.

Requirements:

- Follow **PEP 8 coding standards**.
- Use meaningful variable and function names.
- Organize the logic into appropriate functions.
- Add proper comments and a clear docstring.
- Include basic input validation (e.g., check if the marks list is not empty and contains valid numeric values).
- Do not change the core functionality (calculate total, average, and grade).
- Return only the improved code.

```
marks=[78,85,90,66,88]
```

```
t=0
```

```
for i in marks:
```

```
    t=t+i
```

```
a=t/len(marks)
```

```
if a>=90:
```

```
    print("A")
```

```
elif a>=75:
```

```
    print("B")
```

```
elif a>=60:
```

```
    print("C")
```

```
else:
```

```
print("F")
```

```
#Task 3
def calculate_grade(marks):
    """
    Calculate the total, average, and grade of a student based on their marks.

    Parameters
    -----
    marks : list of int
        A list of marks obtained by the student.

    Returns
    -----
    str
        The grade of the student based on the average marks.
    """
    if not marks:
        return "Error: No marks provided."

    total_marks = sum(marks)
    average_marks = total_marks / len(marks)

    if average_marks >= 90:
        return "Grade: A"
    elif average_marks >= 75:
        return "Grade: B"
    elif average_marks >= 60:
        return "Grade: C"
    else:
        return "Grade: F"

# Example usage
student_marks = [78, 85, 90, 66, 88]
print(calculate_grade(student_marks))
```

```
Grade: B
```

Observation:

The refactored code follows PEP 8 standards with clear function and variable names like `calculate_grade`, `total_marks`, and `average_marks`. It includes a proper docstring explaining parameters and return values, improving readability and documentation. Basic input validation is added to handle empty mark lists, making the program more reliable and user-friendly.

Task 4:

Prompt:

Add a proper docstring and simple inline comments to the following Python function. Do not change the logic. Return only the updated code.

```
def factorial(n):  
    result = 1  
  
    for i in range(1, n + 1):  
        result *= i  
  
    return result
```

```
#Task 4|  
def factorial(n):  
    """  
    Calculate the factorial of a given number.  
  
    Parameters  
    -----  
    n : int  
    | The number for which to calculate the factorial. Must be a non-negative integer.  
  
    Returns  
    -----  
    int  
    | The factorial of the given number.  
  
    Raises  
    -----  
    ValueError  
    | If n is a negative integer.  
    """  
    if n < 0:  
        raise ValueError("Input must be a non-negative integer.")  
  
    result = 1 # Initialize result to 1, as factorial of 0 is 1  
    for i in range(1, n + 1):  
        result *= i # Multiply result by each integer from 1 to n  
  
    return result  
  
# Example usage  
print(factorial(5)) # Output: 120
```

120

Observation:

The function factorial is well-documented with a clear docstring explaining its purpose, parameters, return value, and possible exceptions. It includes input validation to ensure the number is non-negative and raises a ValueError if invalid input is provided. Inline comments are added to explain important steps, improving readability and code clarity.

Task 5:

Improve the following Python password program by adding stronger validation rules (minimum length, uppercase, lowercase, digit, and special character). Refactor it using meaningful names, follow PEP 8, and add a docstring and simple comments. After the code, briefly explain how the improved version is better than the original.

```
#Task 5
import re
def validate_password(password):
    """
    Validate the strength of a password based on multiple security rules.

    Parameters
    -----
    password : str
        The password to be validated.

    Returns
    -----
    str
        A message indicating whether the password is strong or weak.
    """
    if len(password) < 8:
        return "Weak: Password must be at least 8 characters long."

    if not re.search(r'[A-Z]', password):
        return "Weak: Password must contain at least one uppercase letter."

    if not re.search(r'[a-z]', password):
        return "Weak: Password must contain at least one lowercase letter."

    if not re.search(r'[0-9]', password):
        return "Weak: Password must contain at least one digit."

    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
        return "Weak: Password must contain at least one special character."

    return "Strong: Your password is strong."

# Example usage
user_password = input("Enter password: ")
print(validate_password(user_password))
```

```
Enter password: A12345@dfghj
Strong: Your password is strong.
```

Observation:

The enhanced program uses a separate function `validate_password` with clear naming and proper documentation, improving readability and structure. It applies multiple security checks using regular expressions to ensure the password contains uppercase, lowercase, digits, special characters, and a minimum length. Compared to the original version, this implementation is more secure, maintainable, and robust because it enforces real-world password standards and provides clear feedback messages.